TÉCNICO LISBOA

Distributed and Parallel Computing

MPI + OpenMP Hybrid Report

Pedro Rio
MSc Data Science and Engineering
Instituto Superior Técnico
97241

Sebastião Maia Serqueira
MSc Data Science and Engineering
Instituto Superior Técnico
97108

Eduardo Machado
MSc Data Science and Engineering
Instituto Superior Técnico
97133

1 Program Overview

The problem is a matrix-factorisation recommender: a sparse user–item matrix A is approximated by $A \approx L \cdot R$, with L of size *users* × *features* and R of size *features* × *items*. The factors are fitted by gradient descent over the observed entries, and the final dense product $B = L \cdot R$ yields one recommendation per user. The four stages are *ReadInput*, *InitialLR*, *UpdateLR* (repeated for the requested number of iterations), and *FilterFinalMatrix*.

This report covers the **MPI + OpenMP hybrid** deliverable. Its distribution across processes — the 2D grid, the partitioning and the row/column reductions described in the next two sections — is shared with the pure MPI deliverable in `mpi/`, which has its own report. What is specific to this variant is the shared-memory layer inside each rank, in Section 4, and the thread behaviour measured in Section 5.

2 Why a 2D Grid

The simplest MPI design is *replicate-all*: every process keeps full copies of L and R, splits the non-zero list into blocks, computes a partial gradient, and sums it with a global `MPI_Allreduce`. It is correct and short, but it does not scale: the reduced gradient has $O(\mathit{features} \cdot (\mathit{users} + \mathit{items}))$ entries and that volume *does not shrink* as processes are added, while compute per process falls as $1/P$. Communication therefore dominates at scale, and on *wide* instances (a large R) it *anti-scales* — adding processes makes it slower. Memory does not scale either: a single node must hold the whole problem.

Both limits have the same cause — replication — so we distribute the factors over a 2D process grid.

3 The 2D-Grid Version

3.1 Process mesh

`MPI_Dims_create` factors P into a balanced $P_r \times P_c$ grid; `MPI_Cart_create` builds the mesh (with reorder disabled, so a rank's grid id equals its world rank) and `MPI_Cart_sub` derives a *row* and a *column* sub-communicator for each rank.

3.2 Foster methodology

Partitioning. L is split by *user-rows* across the grid rows and R by *item-columns* across the grid columns. A non-zero (u, i) is owned by rank $(\mathit{userBlock}(u), \mathit{itemBlock}(i))$, so each rank holds exactly the L-rows and R-columns its non-zeros touch — memory per rank is $O((\mathit{users} \cdot \mathit{features} + \mathit{features} \cdot \mathit{items})/\sqrt{P})$.

Communication. Each rank computes a partial gradient from its cell's non-zeros (all data lo-

cal), then dL is summed across the *row* communicator and dR down the *column* communicator. These are reductions of $\sqrt{P}$-smaller slices over $\sqrt{P}$-sized sub-communicators, not a global reduce of the whole gradient.

Agglomeration and Mapping. The non-zeros are agglomerated into one contiguous block per grid cell and routed there once with a bucketed `MPI_Scatterv`; one cell maps to one process.

3.3 Deterministic, communication-free init

`initialLR` is seeded with a fixed value, but a partitioned factor cannot simply seek into the random stream (an R-column slice is strided). The trick is *generate-and-discard*: every rank runs the same `srandom(1)` sequence in the serial fill order but stores only the values that fall in its slice. The slices hold exactly the serial values, with no broadcast of the factors.

3.4 Distributed final stage

$B = L \cdot R$, the mask of known entries, and the per-user arg-max are all computed on the cells: each rank forms its B-block, takes a local arg-max over its item columns, and the results are combined across the row with `MPI_MAXLOC` (smallest item index on ties, matching the serial scan) and gathered to the root in user order. There is no serial-root tail.

4 Hybrid OpenMP Layer

Each rank can fan its local work across OpenMP threads. A *single* parallel region is hoisted around the whole iteration loop (the team forks once), `updateLR` uses orphaned worksharing, and the per-thread gradient conflict is resolved with an array reduction — `reduction(+ : dL[:lN], dR[:rN])` — exactly as in the shared-memory version. MPI stays on the main thread under `MPI_THREAD_FUNNELED`: the two `Allreduce` calls run inside an `omp master` block, fenced by barriers. The default is one thread per rank (pure MPI); threads are explicit opt-in through `OMP_NUM_THREADS`, and one thread reproduces the pure-MPI behaviour exactly.

5 Evaluation

The question this deliverable has to answer is not whether it is faster than serial — it is — but whether putting threads *inside* the ranks beats spending the same cores on more ranks. That is only answerable at equal core counts, so Table 1 compares the hybrid at two threads per rank against the pure MPI deliverable at the same total core count.

Provenance. Both designs were measured in the same run by `compare/scripts/benchmark.py` on an Apple Silicon laptop (macOS 15.6, 10 cores, 8 of them performance cores), three repetitions with the minimum kept, every run verified against the instance's `.out` file. The figures are the committed `compare/data/comparison.hybrid.csv` and `comparison.mpi.csv` and are reproducible from this repository.

Table 1: Hybrid (2 threads/rank) vs. pure MPI at equal core count (time, s). Both measured on this laptop in one run.

	2 cores		4 cores		8 cores	
instance	hybrid	pure	hybrid	pure	hybrid	pure
inst200 (wide)	4.29	**3.94**	4.90	**2.25**	**2.90**	2.94
inst500	5.55	**4.76**	3.72	**2.66**	3.84	**3.30**
inst1000	2.45	**2.22**	1.66	**1.23**	1.47	**0.99**

On a single node, pure MPI wins almost everywhere. The hybrid is 9–17 % slower at two cores, 35–118 % slower at four, and 16–48 % slower at eight; the single exception is the wide instance at eight cores, where the two are level (2.90 against 2.94 s, a 1.6 % difference that is inside this machine's run-to-run spread). That is the expected result and worth stating plainly: within one node MPI already communicates through shared memory, so a rank boundary costs almost nothing, while a thread boundary costs a barrier and a reduction on every iteration.

The one place threads nearly pay for themselves is also the informative one. The wide instance (200 users, 10,000 items) has the most lopsided grid, so adding ranks adds communication faster than it adds useful work; there, trading ranks for threads stops being a loss. That is the shape of problem — and, on a cluster, the network cost — that the hybrid exists for.

What this measurement cannot show. Every figure here is from one shared-memory node, which is precisely the setting where the hybrid has least to offer. Its case is one rank per node or socket with threads within, so that inter-node `Allreduce`s run over a $\sqrt{P}$-smaller grid and intra-node work never touches the network. Nothing in this repository can measure that, and no figure above should be read as evidence about it either way.

6 Implications for Parallel and Distributed Computing

6.1 Distribute the data, not just the work

Replicate-all distributes the *work* but not the *data*, so it hits both a memory wall and a communication ceiling. The 2D grid distributes the data: the factor slices and the gradient fall as $1/\sqrt{P}$, and the dense B built in the final stage — which dominates the footprint on every instance here — falls as $1/P$. In this variant there is one more term the pure version does not pay: the gradient loop reduces into per-thread private copies, so each thread adds another $(U{\cdot}F + F{\cdot}I)/\sqrt{P}$ to its rank. Threads buy parallelism with memory, which is worth knowing before raising the thread count on a tight node.

6.2 Match the collective to the decomposition

The win is not a faster `Allreduce`; it is a *smaller* one, over a *smaller* communicator. Because a user's updates are spread only across its grid row and an item's only down its grid column, the global reduction decomposes into independent row and column reductions. Choosing the communicator to match the data layout is what turns flat communication into communication that scales.

6.3 Remove the serial tail

Computing B and the arg-max on the root would be an $O(\mathit{users} \cdot \mathit{items} \cdot \mathit{features})$ sequential tail that, by Amdahl's law, caps the speedup. Distributing it over the same grid keeps the whole pipeline parallel.

6.4 The hybrid is for the memory hierarchy

One rank per node (or socket) × threads-within keeps intra-node work in shared memory and shrinks the process grid, so the inter-node `Allreduce`s are cheaper. On a single node, where MPI already uses shared-memory transport, the hybrid is a wash; its value appears across the network, which is exactly where the cost model says fewer, fatter ranks help.

6.5 The thread count cannot be guessed at runtime

The best `OMP_NUM_THREADS` depends on memory bandwidth, NUMA, the interconnect, and the problem shape — machine-specific quantities a single run cannot know. (A formula $T \approx \sqrt{nnz/(uLocal + iLocal)}$ was measured 20–53 % off the best in half the cases.) The program therefore defaults to one thread and treats threads as explicit configuration: set the rank/thread split in the job script and confirm it with a short scaling sweep.

6.6 Reproducibility under reordering

The partitioned reductions sum the gradient in a different order than the serial code, so floating-point results differ in the last bits — but the arg-max is robust to that, so the recommendations match the serial output bit-for-bit at every process count, grid shape, and thread count. A deterministic serial reference is what makes this verifiable.

The guarantee is about the *output*, not the arithmetic. With threads on, L and R differ in their last bits between runs: OpenMP does not specify the order in which per-thread reduction privates are merged, and implementations merge them as threads finish. Measured here, a fixed configuration produced eight distinct L/R checksums in eight runs — under `schedule(static)` and `schedule(guided)` alike, so the loop schedule does not control it. The recommendations are integer arg-max indices and are insensitive to drift that small, which is why the byte-for-byte guarantee survives. A bitwise-reproducible gradient would need a fixed-order reduction.